

GPS 기반 ETA 예측 · 출발 추천 기능 계획서

작성일: 2026-08-23 (2026-08-23 갱신: 1단계 구현 완료 + 실제 Flutter 앱과 연동) 대상 저장소: **directions-api** (백엔드) + **directions-flutter** (앱)

⚠ **경로 주의:** 이 저장소(**directions-api**)는 **C:\Users\codin\Documents\길 찾기\01_프로젝트**와 **C:\Users\codin\Documents\roadfinder\01_project**에 동일한 **git** 저장소가 두 곳에 체크아웃되어 있다 (경로에 한글이 섞이면 Flutter/Gradle 툴체인이 깨져서, ASCII 전용 경로인 roadfinder 쪽이 실제 개발용으로 쓰이는 것으로 보인다). 이 문서와 백엔드 코드 변경은 **길 찾기** 경로 클론에서 이뤄졌다 — **커밋해서 양쪽에 반영하거나, 최소한 백엔드는 항상 이 클론에서 실행해야** 아래 API가 실제로 존재한다.

앱 쪽은 처음에 **길 찾기\01_프로젝트\directions-flutter**에 빈 프로젝트만 있는 줄 알고 새로 스캐폴딩했었으나, **진짜 앱은 roadfinder\01_project\directions-flutter**에 이미 **Riverpod + Freezed + go_router + Dio** 기반으로 **상당히 진행되어 있었다** (로그인/회원가입/ 기록하기/실시간 추적/AI분석 화면까지 UI가 존재, data/domain 레이어만 비어있었음). 착오로 만든 스캐폴딩은 삭제했고, 실제 앱(roadfinder 경로)에 인증 + GPS 수집 기능을 연결했다. **다만 이 directions-flutter 프로젝트는 git 저장소가 아니다(.git 없음)** — 지금까지의 모든 작업(이번 세션 이전 것 포함)이 버전 관리 없이 디스크에만 존재한다.

진행 상황: 1단계(인증 + GPS 수집 파이프라인 뼈대)를 구현 완료. **users/gps_trips/gps_points** 테이블, **/auth/*./gps/*** API를 만들고, 실제 앱 (roadfinder 경로)의 로그인/회원가입/기록하기/실시간 추적 화면을 여기에 연결했다. 상세는 8절 "구현 현황" 참고. 이후 절의 "초안" 내용은 실제 구현에 맞춰 갱신되었다.

1. 배경 및 목표

현재 **directions-api**는 출발지/도착지를 받아 T-map(보행)·Naver Directions(차량) 경로를 조회하고, 경로 주변 신호등(**signals** 테이블)의 예상 대기시간을 더해 ETA를 보정하는 정적(static) 길찾기 서비스다 (**app/routers/directions.py**). 이동 거리는 2km 이내 근거리 보행으로 제한되어 있다 (**MAX_ROUTE_DISTANCE_M**).

이번 확장의 목표는, 앱이 실제 이동 중 수집한 **GPS 궤적**을 서버로 전송하면 이를 분석해

- 사용자가 실제로 멈췄던 지점(신호 대기, 정차 등)을 탐지하고
- 구간별 실제 이동 속도·거리를 계산하고
- 이 실측 데이터를 학습 데이터로 삼아 **개인화된 ETA(도착 예정 시각)** 를 예측하고
- 목표 도착 시각 대비 **정시 도착 확률**을 계산해
- "여유 있는 출발 / 정시 출발 / 늦어도 지금은 출발" 같은 **출발 타이밍 추천**을 제공하는 것.

즉 기존 기능이 "정적 지도 API 기반 예상 소요시간"이라면, 신규 기능은 "실측 이동 이력 기반 개인화 예측"으로, 두 값을 함께 써서 추천 정확도를 높인다.

2. 전체 파이프라인

```
[Flutter 앱]
  위치 권한 + 백그라운드 트래킹
  → 로컬 버퍼링 (5~10초 간격) → 배치 업로드 (30초~1분마다 또는 이동 종료 시)
    | POST /gps/trips, POST /gps/trips/{id}/points
    ▼
```

[FastAPI: 수집 계층]

gps_points 원시 테이블 적재 (trip_id, user_id, lat/lng, speed, accuracy, ts)



[전처리 파이프라인] (trip 종료 시 또는 배치 잡으로 비동기 실행)

1) 노이즈 제거

- accuracy(정확도 반경) 임계값 초과 포인트 제거
- 물리적으로 불가능한 순간속도(예: 도보 > 20km/h) 제거
- 중복/정지 지터(jitter) 포인트 스무딩 (이동평균 또는 칼만 필터)

2) ST-DBSCAN 정지 지점 클러스터링

- 공간 반경 eps_space, 시간 반경 eps_time, minPts 로 "일정 반경 내에서 일정 시간 이상 머문" 포인트 군집을 정지 클러스터로 탐지
- 각 클러스터 = 정지 구간(체류 시작/종료 시각, 중심 좌표, 체류시간)

3) 속도/거리 계산

- 정지 클러스터 사이 구간을 "이동 구간"으로 분리
- 구간별 이동거리(Haversine 누적), 이동시간, 평균/최대 속도 계산
- 총 이동시간 대비 정지시간 비율 산출



[Feature 저장 / trip_segments, stop_clusters 테이블]



[ETA 예측 모델]

- Baseline: XGBoost (정형 피쳐: 거리, 시간대, 요일, 과거 평균속도, signals 테이블 기반 신호등 개수/예상 대기시간, 최근 N회 동일 경로 실적 등)
- 확장: LSTM (raw/segment 시계열 기반, 궤적 패턴 학습 - 데이터 축적 후 도입)
- 출력: 예상 도착 소요시간의 분포(또는 quantile) → 목표 시각 대비 정시 도착 확률 $P(\text{도착} \leq \text{목표시각})$



[출발 추천 로직]

P(정시 도착) 구간별로 상태 매핑

- $P \geq 0.9$ → "여유 있는 출발"
- $0.9 > P \geq 0.6$ → "정시 출발"
- $0.6 > P \geq 0.3$ → "늦어도 지금은 출발"
- $P < 0.3$ → "이미 늦음 / 다른 수단 고려"

(임계값은 실측 데이터 축적 후 보정 필요 - 1차는 가정치)



[Flutter 앱에 응답] → 알림/위젯으로 표시

3. 데이터베이스 스키마

기존 Base(SQLAlchemy declarative, `app/database.py`)와 `signals` 테이블의 PostGIS 패턴을 그대로 따른다.

`User/GpsTrip/GpsPoint`는 1단계에서 구현 완료 (`app/models.py`). `user_id`는 애초 계획한 기기 UUID 대신, 신규로 추가한 이메일/비밀번호

- JWT 인증 체계의 `users.id(FK)`를 사용한다 — 인증 없이 클라이언트가 스스로 신원을 주장하게 두면 다른 사용자의 trip에 데이터를 주입할 수 있는 문제가 있어, 서버가 토큰으로 검증한 사용자만 자신의 trip에 쓰도록 했다. `StopCluster/TripSegmentFeature`는 2~3단계 (전처리/피쳐 계산)에서 아직 구현되지 않은 초안이다.

app/models.py - 구현 완료

```
class User(Base):
    """이메일/비밀번호 기반 사용자 계정."""
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    email: Mapped[str] = mapped_column(String(255), nullable=False,
unique=True, index=True)
    username: Mapped[str] = mapped_column(String(64), nullable=False,
unique=True, index=True)
    hashed_password: Mapped[str] = mapped_column(String(255), nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())

class GpsTrip(Base):
    """앱이 하나의 이동(출발~도착)을 시작~종료할 때 생성하는 단위."""
    __tablename__ = "gps_trips"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    user_id: Mapped[int] = mapped_column(Integer, ForeignKey("users.id"),
nullable=False, index=True)
    label: Mapped[str] = mapped_column(String(100), nullable=True) # 앱 "기록
하기" 화면의 기록 이름
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
server_default=func.now())
    ended_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=True)
    origin_lat: Mapped[float] = mapped_column(Float, nullable=True)
    origin_lng: Mapped[float] = mapped_column(Float, nullable=True)
    dest_lat: Mapped[float] = mapped_column(Float, nullable=True)
    dest_lng: Mapped[float] = mapped_column(Float, nullable=True)
    target_arrival_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
nullable=True)
    status: Mapped[str] = mapped_column(String(16), nullable=False,
default="active")
    # status: active(수집 중) / completed(종료) / discarded(폐기)

class GpsPoint(Base):
    """원시 GPS 포인트 (노이즈 제거 전). 좌표는 EPSG:4326(WGS84)."""
    __tablename__ = "gps_points"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    trip_id: Mapped[int] = mapped_column(Integer, ForeignKey("gps_trips.id"),
nullable=False, index=True)
    geom: Mapped[object] = mapped_column(Geometry(geometry_type="POINT",
srid=4326), nullable=False)
    speed_mps: Mapped[float] = mapped_column(Float, nullable=True)
    accuracy_m: Mapped[float] = mapped_column(Float, nullable=True)
    recorded_at: Mapped[datetime] = mapped_column(DateTime(timezone=True),
```

```

nullable=False)
    is_noise: Mapped[bool] = mapped_column(Boolean, nullable=False,
default=False)

```

원래 초안에서는 `GpsTrip.origin_geom/dest_geom`을 PostGIS `Geometry` 컬럼으로 뒀지만, 현재 trip 출발/도착지에 대한 공간 쿼리(예: 반경 검색)가 필요 없어 단순 `Float` 위경도 쌍으로 구현을 단순화했다. 추후 "과거 동일 경로 검색" 같은 기능이 필요해지면 그때 PostGIS 컬럼으로 옮기면 된다.

app/models.py 에 추가 예정 (2~3단계, 아직 미구현)

```

class StopCluster(Base):
    """ST-DBSCAN으로 탐지된 정지 구간."""
    __tablename__ = "stop_clusters"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    trip_id: Mapped[int] = mapped_column(Integer, ForeignKey("gps_trips.id"),
index=True)
    center_geom: Mapped[object] = mapped_column(Geometry("POINT", srid=4326))
    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    ended_at: Mapped[datetime] = mapped_column(DateTime(timezone=True))
    duration_s: Mapped[int] = mapped_column(Integer)
    matched_signal_id: Mapped[int] = mapped_column(ForeignKey("signals.id"),
nullable=True)

class TripSegmentFeature(Base):
    """ETA 모델 학습/추론용 trip 단위 피쳐 + 실측 라벨."""
    __tablename__ = "trip_segment_features"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    trip_id: Mapped[int] = mapped_column(Integer, ForeignKey("gps_trips.id"),
unique=True)
    distance_m: Mapped[float]
    moving_time_s: Mapped[int]
    stopped_time_s: Mapped[int]
    stop_count: Mapped[int]
    avg_speed_mps: Mapped[float]
    signal_count_on_route: Mapped[int] = mapped_column(nullable=True)
    hour_of_day: Mapped[int]
    day_of_week: Mapped[int]
    actual_duration_s: Mapped[int] = mapped_column(nullable=True) # 학습 라벨

```

`StopCluster.matched_signal_id`로 기존 `signals`(신호등) 테이블과 연결해, "이 정지가 실제 신호등 대기였는지" 판별할 수 있게 한다 — 기존 `ST_DWithin` 매칭 로직을 재사용.

4. API 엔드포인트

`app/routers/auth.py`, `app/routers/gps.py`가 구현 완료. `/eta/`는 3~4단계에서 피쳐 계산 모델이 준비된 뒤 추가할 예정.

Method	Path	설명	상태
POST	/auth/register	이메일/비밀번호 회원가입 → 액세스 토큰 발급	완료
POST	/auth/login	로그인 → 액세스 토큰 발급	완료
GET	/auth/me	현재 로그인 사용자 정보	완료
POST	/gps/trips	trip 시작 (label, origin, target_arrival_at 등록) → trip_id 반환. 로그인 필요	완료
GET	/gps/trips	로그인한 사용자의 trip 목록 (최신순). 앱의 "이전 기록 불러오기"용	완료
GET	/gps/trips/{id}	trip 상태 조회 (본인 소유만)	완료
POST	/gps/trips/{id}/points	GPS 포인트 배치 업로드 (앱이 주기적으로 호출)	완료
POST	/gps/trips/{id}/finish	trip 종료 → 현재는 point 개수 집계까지만. 전처리·클러스터링·피쳐 계산은 미구현	완료(수집만)
GET	/eta/predict	현재 위치 + 목적지 + 목표 도착시각 → 예상 소요 시간, 정시 도착 확률	미구현 (4단계)
GET	/eta/departure-recommendation	위 확률 기반 출발 상태 문구 반환	미구현 (5단계)

/gps/* 엔드포인트는 모두 `Authorization: Bearer <token>` 필요. trip 소유자가 아니면 403을 반환한다 (app/routers/gps.py의 `_get_own_trip_or_404`).

5. ML 파이프라인 설계

1차 (baseline, XGBoost)

- 입력 피쳐: 직선/경로 거리, 요일·시간대, 신호등 개수 및 예상 대기시간 (기존 `signal_delay_estimate_s`로 직 재사용), 사용자별 동일/유사 경로 과거 평균 속도, 최근 실시간 교통 보정 계수 (`RealtimeTraffic.eta_factor`, driving 모드).
- 라벨: 실측 `actual_duration_s`.
- 장점: 적은 데이터로도 학습 가능, 해석 가능(feature importance), 콜드스타트에 강함.

2차 (확장, LSTM)

- 입력: 정규화된 GPS 궤적 시퀀스(구간별 속도·정지 패턴) — trip 내부 시계열 처리
- 목적: XGBoost가 못 잡는 "사람마다 다른 이동 습관(예: 신호 무시하고 빨리 건너는지, 중간에 매번 들르는 곳이 있는지)" 같은 시퀀스 패턴 학습.
- 데이터가 충분히 쌓이기 전(사용자당 최소 수십 회 이동 기록)에는 XGBoost 단독 사용, 이후 앙상블 또는 대체.

정시 도착 확률 산출

- 회귀 모델의 점추정치만으로는 확률을 못 구하므로, quantile regression (XGBoost `reg:quantileerror` 또는 별도 분위수 모델) 또는 예측 오차의 경험적 분포를 이용해 $P(\text{실제 소요시간} \leq \text{남은시간})$ 을 근사한다.

콜드스타트 대응

- 신규 사용자/신규 경로: 개인 이력이 없으면 population 모델(전체 사용자 평균) → 이력이 쌓이면 개인화 모델로 점차 가중치 이동 (예: 베이지안 shrinkage 또는 간단한 가중평균 $w * \text{personal} + (1-w) * \text{population}$, w 는 이력 횟수에 비례).

6. Flutter 앱 요구사항

`directions-flutter`는 `flutter create`로 새로 스캐폴딩하고 최소 트래킹 기능을 구현 완료 (아래 "구현 현황" 참고). 남은 항목:

- ☒ 위치 권한: 포그라운드(`geolocator`) — 완료
- ☐ 백그라운드 트래킹(Android `ACCESS_BACKGROUND_LOCATION`, iOS `Always` 권한, `flutter_background_service` 등) — 미구현. 앱이 화면에 떠 있을 때만 수집된다.
- ☒ 배치 업로드 (20개 모이거나 30초마다) — 완료, `LocationTracker` 참고
- ☐ 로컬 큐잉/재시도: 지금은 업로드 실패 시 메모리 버퍼에만 되돌리므로 앱이 종료되면 유실된다. SQLite/Hive 영속 큐는 미구현.
- 배터리 최적화: `distanceFilter: 5m`로 불필요한 업데이트는 억제했지만, 정지 감지 시 간격을 더 늘리는 적응형 로직은 아직 없음.
- ☐ 목표 도착 시각 입력 UI, ETA/추천 결과 표시 화면, 알림 — `/eta/*` API가 없어서 아직 미구현.

7. 단계별 로드맵

1. ☒ 데이터 수집 파이프라인 — `User/GpsTrip/GpsPoint` 테이블, 인증(JWT) + 업로드 API, Flutter 로그인/추적 화면까지 구현 완료. 실 데이터 축적을 시작할 수 있는 상태. (백그라운드 트래킹·로컬 큐잉은 남음 — 6절 참고)
2. 전처리 + ST-DBSCAN — 노이즈 제거, 정지 클러스터링 배치 잡, `StopCluster` 저장, 기존 `signals` 테이블과 매칭
3. 속도/거리 계산 + 피쳐 저장 — `TripSegmentFeature` 생성 로직
4. ETA baseline (XGBoost) — 학습 스크립트, 모델 아티팩트 저장/서빙, `/eta/predict` 엔드포인트
5. 출발 추천 로직 — 확률 임계값 매핑, `/eta/departure-recommendation`, 앱 알림 연동
6. LSTM 고도화 — 데이터량 확보 후 순차 착수, XGBoost와 비교/양상불

데이터가 없으면 3~5단계 모델이 무의미하므로, 1~2단계를 먼저 끝내고 최소 몇 주간 실사용 데이터를 모으는 기간을 계획에 넣는 것을 권장한다.

8. 기술 스택 추가 항목

- `requirements.txt` 추가 후보: `scikit-learn`(전처리/거리 계산 보조), `xgboost`, `numpy`, `pandas`. ST-DBSCAN은 표준 라이브러리가 마땅치 않아 `scikit-learn`의 DBSCAN을 3차원(위경도+정규화된 시간축) 입력으로 응용하거나 직접 구현.
- LSTM 단계 도입 시 `torch` 또는 `tensorflow` 추가 (모델 서빙 방식 별도 검토 — FastAPI 프로세스 내 추론 vs 별도 추론 서버).
- 모델 학습은 API 프로세스와 분리(배치 스크립트/Celery/cron)해서 API 응답 지연을 일으키지 않도록 한다.

9. 확인이 필요한 사항

- ~~캐안화~~범위 → 결정됨: 익명 기기 UUID 대신 이메일/비밀번호 + JWT 인증을 먼저 구현하고, `GpsTrip.user_id`는 인증된 `users.id`를 사용하기로 함.
- 목표 도착 시각 입력 주체: 사용자가 직접 입력("9시까지 도착")하는 방식으로 가정. 캘린더 연동 등은 범위 밖으로 둬. (`GpsTripCreate.target_arrival_at`으로 받는 필드는 이미 구현했지만, 앱 UI에서 입력받는 화면은

아직 없음)

- **실시간 연속 트래킹 vs 단발성 조회:** "이동 중 계속 추적하며 추천 갱신"까지 포함할지, 아니면 "출발 전 한 번 조회"만 할지에 따라 앱 백그라운드 트래킹 복잡도가 크게 달라짐. 위 계획은 이동 전체를 추적하는 것을 전제로 하되, 현재 구현은 포그라운드 추적까지만 됨.
- **위치 데이터 보관/프라이버시:** 원시 GPS는 개인 이동 패턴을 드러내는 민감 정보이므로 보관 기간, 익명화, 사용자 삭제 요청 처리 방침을 정해야 함 (학원 프로젝트 범위에서는 최소한 "탈퇴 시 삭제" 정도는 명시 권장). 아직 미구현 — 회원 탈퇴 API와 함께 처리 필요.

10. 1단계 구현 현황 및 실행 방법

백엔드 (directions-api)

- 추가 파일: `app/auth.py`(비밀번호 해시-JWT), `app/routers/auth.py`, `app/routers/gps.py`, `app/models.py/app/schemas.py`에 관련 모델·스키마 추가.
- `requirements.txt`에 `bcrypt`, `PyJWT`, `email-validator` 추가.
- 검증: 로컬 `venv`에 의존성 설치 후 앱 임포트·라우트 등록·OpenAPI 스키마 생성 확인, `hash_password/verify_password/create_access_token`의 JWT 왕복까지 단위 테스트 완료. 단, 이 환경에는 **Docker와 로컬 PostGIS가 없어 실제 DB에 연결한 end-to-end 테스트(회원가입→trip 생성→포인트 업로드)는 못 했다.** 아래 명령으로 직접 확인 권장:

```
docker compose -f docker-compose.dev.yml up
# 이후 http://localhost:8000/docs 에서 /auth/register → /gps/trips →
/gps/trips/{id}/points 순서로 테스트
```

앱 (directions-flutter)

- `flutter create`로 신규 스캐폴딩 (Android/iOS). `geolocator`, `http`, `shared_preferences` 의존성 추가.
- 화면: `LoginScreen`(회원가입/로그인) → `TrackingScreen`(이동 시작/종료, 실시간 수집 개수 표시).
- `dart analyze` 통과, 위젯 테스트 1개 통과, `flutter build apk --debug` 로 실제 **APK 빌드까지 성공 확인.**
- 주의: 프로젝트 경로에 한글/공백(`01_프로젝트`)이 포함돼 있어 Gradle이 기본적으로 빌드를 거부한다. `android/gradle.properties`에 `android.overridePathCheck=true`를 추가해 우회했다 — 가능하면 추후 ASCII 전용 경로로 옮기는 것이 더 안전하다.
- API 주소는 `lib/api_config.dart`의 `kApiBaseUrl`(기본값 `http://10.0.2.2:8000`, Android 에뮬레이터에서 PC의 localhost를 가리킴)이며, `flutter run --dart-define=API_BASE_URL=http://<PC-IP>:8000`으로 실제 기기에서 바꿔 실행 가능.

11. 다음 액션

1. 위 명령으로 백엔드 DB 연동을 직접 확인 (Docker 실행 후 `/docs`에서 수동 테스트).
2. 실기기/에뮬레이터에서 `flutter run`으로 로그인→이동 시작→GPS 수집→이동 종료 흐름을 눈으로 확인.
3. 문제 없으면 2단계(노이즈 제거 + ST-DBSCAN 정지 클러스터링) 착수.